

Shortest Paths: BFS, Dijkstra and 0-1 BFS

Algorithm Design & Analysis Tutorial

August 19, 2026

Warm-up

Let $G = (V, E)$ be an undirected graph with positive edge weights, and let $\pi(s, t)$ be a shortest path between two vertices $s, t \in V$. Will $\pi(s, t)$ still be a shortest path if

1. the weight of every edge is multiplied by a fixed constant $c > 0$?
2. the weight of every edge is incremented by a fixed constant $c > 0$?

A yes needs an argument that covers every competing s - t path. A no needs one explicit counterexample. In both cases, say where the hypothesis $c > 0$ is used.

Part 1: multiplying every weight by $c > 0$

Write $w'(e) = c w(e)$.

Claim

$\pi(s, t)$ is still a shortest s - t path, and $\delta'(u, v) = c \delta(u, v)$ for every pair u, v .

Proof. Scaling changes neither V nor E , so G and G' have exactly the same s - t paths: the competitors are the same objects in both graphs. For any such path P ,

$$w'(P) = \sum_{e \in P} c w(e) = c w(P).$$

Let Q be an arbitrary s - t path. Since $\pi(s, t)$ is shortest in G we have $w(\pi) \leq w(Q)$, and multiplying an inequality by $c > 0$ preserves its direction, so

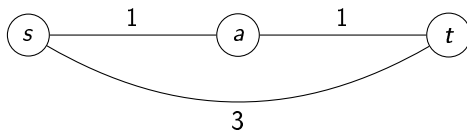
$$w'(\pi) = c w(\pi) \leq c w(Q) = w'(Q).$$

As Q was arbitrary, $\pi(s, t)$ has minimum weight in G' too, and taking the minimum over all s - t paths gives $\delta'(s, t) = c \delta(s, t)$. Nothing used anything specific about s and t . ■

For $c < 0$ the middle inequality reverses, so the hypothesis is doing real work.

Part 2: adding $c > 0$ to every weight

Now $w'(e) = w(e) + c$, so a path Q with $|Q|$ edges has $w'(Q) = w(Q) + c|Q|$. The shift is not common to all competitors: it grows with the edge count, so paths with few heavy edges gain on paths with many light edges and the ranking can flip.



Here $w(s a t) = 2 < 3 = w(s t)$, so $\pi(s, t) = s a t$. Take $c = 2$:

$$w'(s a t) = 3 + 3 = 6, \quad w'(s t) = 5.$$

The direct edge is now the unique shortest path, so the answer is no.

Exactly one line of Part 1 fails: $w'(Q) = c w(Q)$ becomes $w(Q) + c|Q|$, which is a common shift only among paths with equally many edges. If all s - t paths had the same edge count, the answer would be yes again.

Setting

$G = (V, E)$, weights $w : E \rightarrow \mathbb{R}_{\geq 0}$, source s , and

$$\delta(v) = \min\{ w(P) : P \text{ is an } s \rightsquigarrow v \text{ path} \}, \quad \delta(v) = \infty \text{ if } v \text{ is unreachable.}$$

- ▶ **Optimal substructure.** Every subpath of a shortest path is a shortest path, by cut and paste.
- ▶ Since $w \geq 0$, deleting a cycle never increases weight, so a minimum-weight walk can always be taken to be a simple path.
- ▶ All three algorithms today keep an estimate $d[v]$, start from $d[s] = 0$ and $d[v] = \infty$, and only ever run $\text{Relax}(u, v)$: if $d[u] + w(u, v) < d[v]$ then $d[v] \leftarrow d[u] + w(u, v)$.
- ▶ **Upper-bound property.** Relaxation only ever sets $d[v]$ to the weight of an actual $s \rightsquigarrow v$ walk, so $d[v] \geq \delta(v)$ at all times and $d[v]$ never increases.

The plan

The three algorithms run the same loop and differ only in the structure holding the frontier. We prove the greedy step once, for Dijkstra, then only check that the other structures return a minimum key.

Unit weights: BFS

All weights 1, so cheapest means fewest edges and the frontier is a FIFO queue.

```
1 d[s] = 0; q.push(s);
2 while (!q.empty()) {
3     int u = q.front(); q.pop();
4     for (int v : adj[u])
5         if (d[v] == INF) { d[v] = d[u] + 1; q.push(v); } // v seen for the first time
6 }
```

Invariant

If the queue is $\langle v_1, \dots, v_r \rangle$ front to back, then $d[v_1] \leq \dots \leq d[v_r] \leq d[v_1] + 1$.

Proof. Initially the queue is $\langle s \rangle$. In the iteration that pops v_1 , every pushed vertex x gets $d[x] = d[v_1] + 1$, which is at least $d[v_r]$ so the order survives, and at most $d[v_2] + 1$ so the bound on the last element survives. ■

So the front always holds a minimum key: BFS is Dijkstra with a queue in place of the heap, and correctness follows from the next slide. Cost $O(|V| + |E|)$.

Arbitrary non-negative weights: Dijkstra

Extract the vertex with the smallest estimate, declare it final, relax its outgoing edges.

```
1  typedef pair<long long,int> pli; // (distance, vertex)
2
3  vector<long long> dijkstra(int n, vector<vector<pli> >& adj, int s) {
4      vector<long long> dist(n, LLONG_MAX);
5      vector<bool> done(n, false);
6      priority_queue<pli, vector<pli>, greater<pli> > pq;
7      dist[s] = 0; pq.push({0, s});
8
9      while (!pq.empty()) {
10         int u = pq.top().second; pq.pop();
11         if (done[u]) continue; // stale copy of u
12         done[u] = true; // dist[u] is final here
13
14         for (pli e : adj[u]) { // e = (weight, neighbour)
15             long long w = e.first; int v = e.second;
16             if (dist[u] + w < dist[v]) {
17                 dist[v] = dist[u] + w;
18                 pq.push({dist[v], v});
19             }
20         }
21     }
22     return dist;
23 }
```

Dijkstra: correctness and cost

Theorem

Whenever a vertex u is finalised, $d[u] = \delta(u)$.

Proof. Suppose not and let u be the first vertex finalised with $d[u] \neq \delta(u)$; by the upper-bound property $d[u] > \delta(u)$, so $\delta(u) < \infty$ and $u \neq s$. Fix a shortest path $P : s \rightsquigarrow u$, let y be the first vertex of P not yet finalised, and let x be its predecessor on P (possibly $x = s$).

Since x was finalised earlier and u is the first violation, $d[x] = \delta(x)$. Relaxing (x, y) at that moment gave

$$d[y] \leq d[x] + w(x, y) = \delta(x) + w(x, y) = \delta(y),$$

the last step by optimal substructure, so $d[y] = \delta(y)$. Now y lies on P and all weights are non-negative, hence $\delta(y) \leq \delta(u)$; and y was still in the queue when u was extracted with the minimum key, hence $d[u] \leq d[y]$. Chaining,

$$d[u] \leq d[y] = \delta(y) \leq \delta(u) < d[u],$$

a contradiction. ■

Non-negativity was used in exactly one place, $\delta(y) \leq \delta(u)$, and nothing about the heap was used beyond “the extracted key is minimum”. With $s \rightarrow u$ of weight 1, $s \rightarrow v$ of weight 2 and $v \rightarrow u$ of weight -2 , the algorithm finalises $d[u] = 1$ while $\delta(u) = 0$.

Cost. At most $|E|$ pushes at $O(\log |V|)$ each: $O(|E| \log |V|)$ time, $O(|V| + |E|)$ space.

Problem: preferred directions

A country has n cities and m roads. Every road joins two cities and can be driven both ways, but each road comes with a preferred direction of travel. Driving a road along its preferred direction is fine; driving it the other way is allowed but counts as one violation.

Given a source s and a target t , find the minimum number of violations on a route from s to t , or report that t is unreachable. Take $n, m \leq 10^6$.

Nothing here is a distance, but the cost of a route is a sum over the roads it uses, of 0 or 1 per road, and we are minimising that sum. That is a shortest path with weights in $\{0, 1\}$.

The reduction, and why it is exact

Build a directed graph $G' = (V, E')$ on the same n cities. For each road between u and v whose preferred direction is $u \rightarrow v$, put into E'

$$u \rightarrow v \text{ with weight } 0, \quad v \rightarrow u \text{ with weight } 1, \quad \text{so } |E'| = 2m.$$

Claim

The minimum number of violations on a route from s to t equals $\delta_{G'}(s, t)$.

Proof. A route is a sequence of cities $s = c_0, c_1, \dots, c_L = t$ in which consecutive cities are joined by a road. Each step $c_i \rightarrow c_{i+1}$ is either along the preferred direction of that road, which is a weight-0 edge of G' , or against it, which is a weight-1 edge of G' . So routes from s to t and directed walks from s to t in G' are the same objects, and the number of violations of a route is the weight of the corresponding walk.

Minimising over routes is therefore minimising walk weight in G' , and since $w \geq 0$ the minimum over walks equals the minimum over paths, namely $\delta_{G'}(s, t)$. A route exists iff t is reachable from s in G' , iff t is reachable in the underlying undirected graph. ■

From heap to deque

Dijkstra on G' answers this in $O(m \log n)$. Look at what the heap actually holds.

Observation

If u is finalised with $\text{dist}[u] = k$, relaxing its edges creates keys k (through a 0-edge) and $k + 1$ (through a 1-edge), and nothing else. At any moment the queue holds at most two distinct keys.

Two values do not need a heap. Keep the frontier in a deque: weight 0 goes to the front, weight 1 goes to the back, and we always take from the front.

```
1 for (pii e : adj[u]) { // e = (weight, neighbour), weight is 0 or 1
2     int w = e.first, v = e.second;
3     if (dist[u] + w < dist[v]) {
4         dist[v] = dist[u] + w;
5         if (w == 0) dq.push_front({dist[v], v});
6         else dq.push_back ({dist[v], v});
7     }
8 }
```

The rest of the loop is Dijkstra unchanged, done test included. What has to be proved is that `pop_front` returns a minimum key.

The deque invariant

Let the deque hold keys $k_1, \dots, k_r$ from front to back.

Invariant $\mathcal{I}$

$$k_1 \leq k_2 \leq \dots \leq k_r \quad \text{and} \quad k_r \leq k_1 + 1.$$

Initialisation. The deque is the single entry $(0, s)$.

A remark first. If $\mathcal{I}$ holds and we pop a front entry (d, u) with $\text{done}[u]$ false, then $d = \text{dist}[u]$. Indeed $\text{dist}[u] \leq d$ always; if $\text{dist}[u] < d$, the relaxation that produced the smaller value pushed an entry $(\text{dist}[u], u)$, which cannot have been popped yet since popping it would have set $\text{done}[u]$. That entry has key below k_1 , contradicting $\mathcal{I}$.

Maintenance. Pop the front entry, of key $m = k_1$; the surviving keys are sorted and lie in $[m, m + 1]$. If $\text{done}[u]$ we push nothing. Otherwise $\text{dist}[u] = m$ by the remark, so a successful relaxation along an edge of weight w pushes the key $m + w$:

- ▶ $w = 0$: key m goes to the front, and it is at most every surviving key, so the order survives; the minimum is still m and the maximum still at most $m + 1$.
- ▶ $w = 1$: key $m + 1$ goes to the back, and it is at least every surviving key, so the order survives; the maximum is $m + 1$ and the minimum is m , or $m + 1$ if the deque was empty.

Both conditions are restored. ■

0-1 BFS: correctness and cost

Theorem

With weights in $\{0, 1\}$, 0-1 BFS computes $\text{dist}[v] = \delta(v)$ for all v in $O(|V| + |E|)$ time.

Correctness. By $\mathcal{I}$ the keys increase from front to back, so the front entry always carries a minimum key and `pop_front` is an Extract-Min. The rest of the loop is Dijkstra line for line, and the weights are non-negative, so the Dijkstra Theorem applies unchanged. ■

Cost. Each vertex passes the done test once, so each adjacency list is scanned once and there are $O(|E|)$ relaxations. Every push follows a successful relaxation, so at most $O(|E|)$ entries enter the deque and each is popped once, at $O(1)$ per operation. Total $O(|V| + |E|)$ time and space, against $O(|E| \log |V|)$ for the heap.

On the roads problem: build G' , run this from s , answer $\text{dist}[t]$, with $\text{dist}[t] = \infty$ meaning unreachable. Time $O(n + m)$.

Full solution: preferred directions

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef pair<int,int> pii; // (weight, neighbour)
4
5  int main() {
6      int n, m, s, t;
7      scanf("%d %d %d %d", &n, &m, &s, &t);
8      vector<vector<pii> > adj(n + 1);
9      for (int i = 0; i < m; i++) {
10         int u, v; scanf("%d %d", &u, &v); // road u--v, preferred direction u -> v
11         adj[u].push_back({0, v}); // driving u -> v costs no violation
12         adj[v].push_back({1, u}); // driving v -> u costs one violation
13     }
14
15     vector<int> dist(n + 1, INT_MAX);
16     vector<bool> done(n + 1, false);
17     deque<pii> dq; // (key, vertex)
18     dist[s] = 0; dq.push_front({0, s});
19
20     while (!dq.empty()) {
21         int u = dq.front().second; dq.pop_front();
22         if (done[u]) continue;
23         done[u] = true;
24         for (pii e : adj[u]) {
25             int w = e.first, v = e.second;
26             if (dist[u] + w < dist[v]) {
27                 dist[v] = dist[u] + w;
28                 if (w == 0) dq.push_front({dist[v], v});
29                 else dq.push_back ({dist[v], v});
30             }
31         }
32     }
33     printf("%d\n", dist[t] == INT_MAX ? -1 : dist[t]); // -1: t unreachable
34 }
```